NC State University

Department of Electrical and Computer Engineering

ECE 463/563 (Prof. Rotenberg)

Project #1: Cache Design, Memory Hierarchy Design

REPORT TEMPLATE (Version 2.0)

by

<< YOUR NAME HERE >>

NCSU Honor Pledge: "I have neither given nor received unauthorized aid on this project."

Student's electronic signature: _____Madhumitha Ramkumar_____

(sign by typing your name)

Course number: _____563_____

(463 or 563 ?)

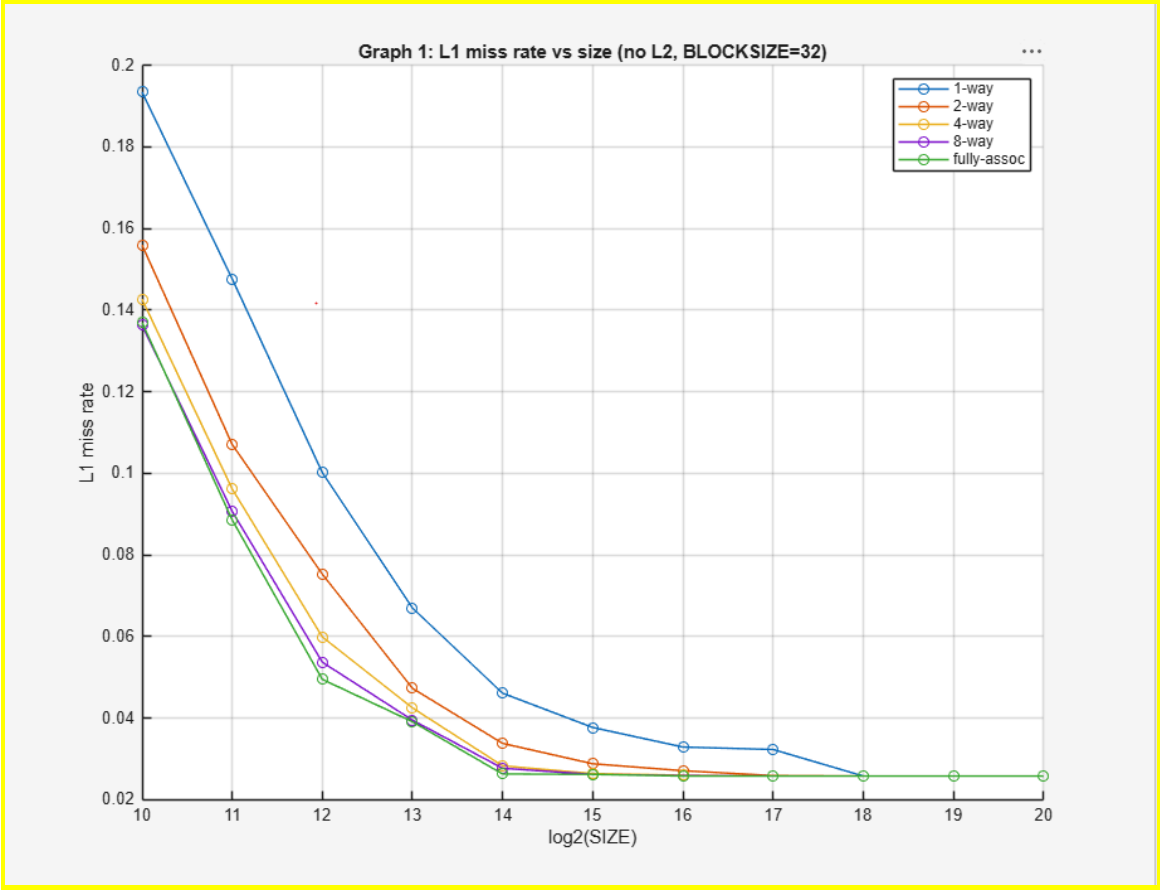
1. L1 cache exploration: SIZE and ASSOC

GRAPH #1 (***total number of simulations: 55***)

For this experiment:

- Benchmark trace: gcc_trace.txt
- L1 cache: SIZE is varied, ASSOC is varied, BLOCKSIZE = 32.
- L2 cache: None.
- Prefetching: None.

Plot L1 miss rate on the y-axis versus $\log_2$(SIZE) on the x-axis, for eleven different cache sizes: SIZE = 1KB, 2KB, … , 1MB, in powers-of-two. (That is, $\log_2$(SIZE) = 10, 11, …, 20.) The graph should contain five separate curves (*i.e.*, lines connecting points), one for each of the following associativities: direct-mapped, 2-way set-associative, 4-way set-associative, 8-way set-associative, and fully-associative. All points for direct-mapped caches should be connected with a line, all points for 2-way set-associative caches should be connected with a line, *etc*.



Answer the following questions:

1. For a given associativity, how does increasing cache size affect miss rate?

Increasing cache size decreases miss rate, with diminishing returns. Capacity and conflict misses fall as the cache grows, but it converges at the compulsory miss rate beyond which increasing capacity provides no benefit.

2. For a given cache size, how does increasing associativity affect miss rate?

Increasing associativity decreases miss rate by reducing conflict misses, but with diminishing returns and the benefit shrinks as cache size grows.

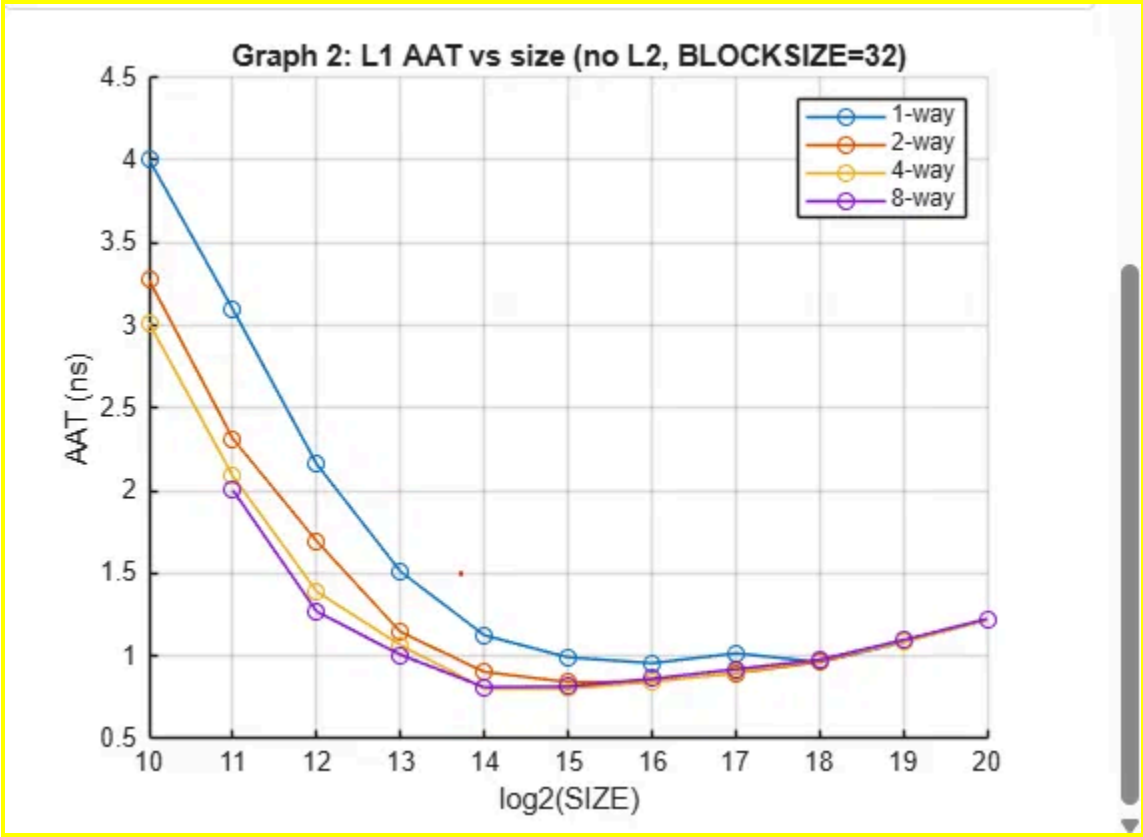
3. Estimate the *compulsory miss rate* from the graph and briefly explain how you arrived at this estimate.

compulsory miss rate = __0.025__
How I arrived at this estimate: On the graph, all five curves converge to the same value and flatten and increasing either size or associativity beyond that point produces no further reduction. So neither remaining knob changes the miss rate, capacity and conflict misses have been eliminated and only compulsory misses remain. That floor is 0.025.

GRAPH #2 (***no additional simulations with respect to GRAPH #1***)

Same as GRAPH #1, except make the following changes:

- The y-axis should be AAT instead of L1 miss rate.
- Do NOT include fully-associative cache configurations (power-hungry, impractical cost for implementing LRU). Thus, GRAPH #2 should contain four (not five) separate curves: direct-mapped, 2-way set-associative, 4-way set-associative, 8-way set-associative.



Answer the following question:

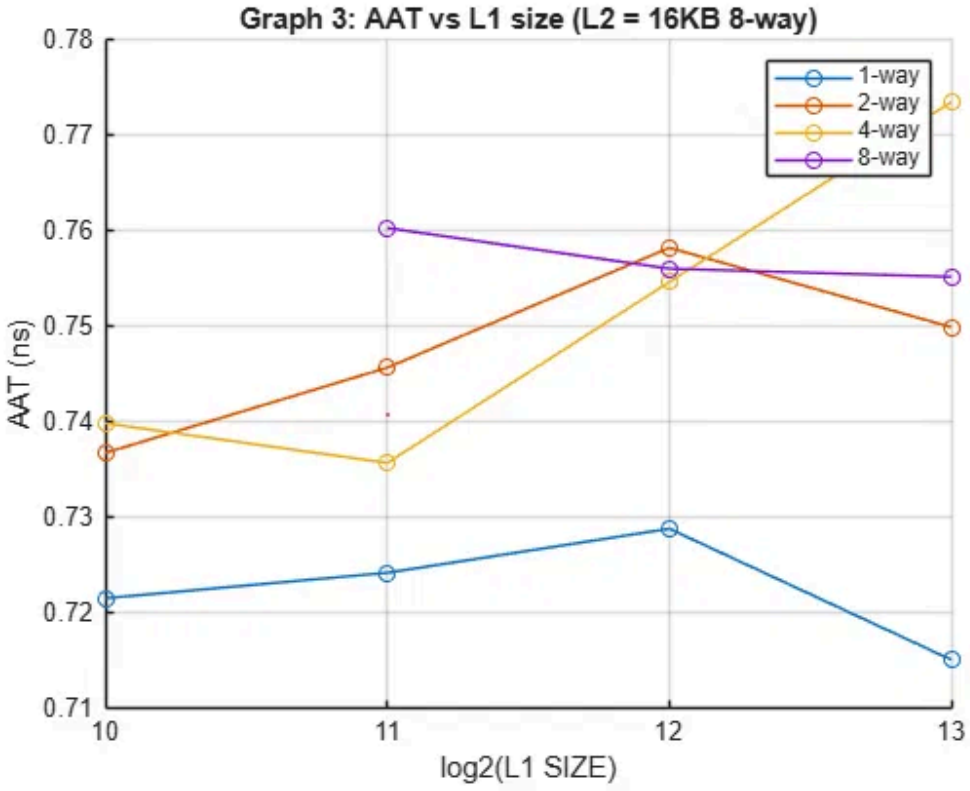
1. For a memory hierarchy with only an L1 cache and BLOCKSIZE = 32, which configuration yields the best (*i.e.*, lowest) AAT and what is that AAT?

Direct-mapped or set-associative cache configuration that yields the lowest AAT:
32 KB, 4-way set-assoc AAT for this configuration: 0.8019 ns

GRAPH #3 (***total number of simulations: 16***)

Same as GRAPH #2, except make the following changes:

- Add the following L2 cache to the memory hierarchy: 16KB, 8-way set-associative, same block size as L1 cache.
- Vary the L1 cache size only between 1KB and 8KB (since L2 cache is 16KB). (*And as with GRAPH #2, GRAPH #3 should contain four (not five) separate curves for L1 associativity: direct-mapped, 2-way set-associative, 4-way set-associative, 8-way set-associative.*)



Answer the following questions:

1. With the L2 cache added to the system, which L1 cache configuration yields the best (*i.e.*, lowest) AAT and what is that AAT?

L1 configuration that yields the lowest AAT with 16KB 8-way L2 added: 8 KB, direct mapped AAT for this configuration: _0.715 ns

2. How does the lowest AAT with L2 cache (GRAPH #3) compare with the lowest AAT without L2 cache (GRAPH #2)?

The lowest AAT with L2 cache is less than the lowest AAT without L2 cache.

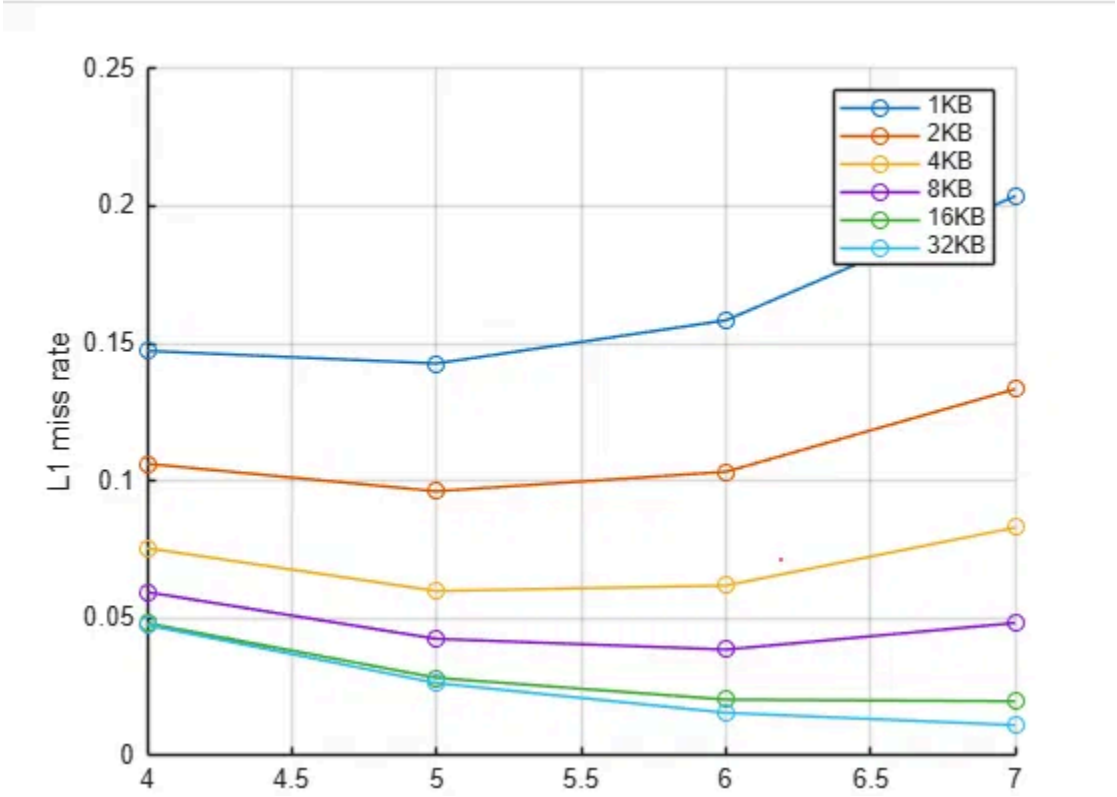
2. L1 cache exploration: SIZE and BLOCKSIZE

GRAPH #4 (***total number of simulations: 24***)

For this experiment:

- Benchmark trace: gcc_trace.txt
- L1 cache: SIZE is varied, BLOCKSIZE is varied, ASSOC = 4.
- L2 cache: None.
- Prefetching: None

Plot L1 miss rate on the y-axis versus $\log_2$(BLOCKSIZE) on the x-axis, for four different block sizes: BLOCKSIZE = 16, 32, 64, and 128. (That is, $\log_2$(BLOCKSIZE) = 4, 5, 6, and 7.) The graph should contain six separate curves (*i.e.*, lines connecting points), one for each of the following L1 cache sizes: SIZE = 1KB, 2KB, …, 32KB, in powers-of-two. All points for SIZE = 1KB should be connected with a line, all points for SIZE = 2KB should be connected with a line, *etc*.



Answer the following questions:

1. Do smaller caches prefer smaller or larger block sizes?

Smaller caches prefer smaller block sizes. For example, the smallest cache considered in Graph #4 (1KB) achieves its lowest miss rate with a block size of __32__ B.

2. Do larger caches prefer smaller or larger block sizes?

Larger caches prefer larger block sizes. For example, the largest cache considered in Graph #4 (32KB) achieves its lowest miss rate with a block size of __128__ B.

3. As block size is increased from 16 to 128, is the tension between *exploiting more spatial locality* and *cache pollution* evident in the graph? Explain.

Yes, the tension between *exploiting more spatial locality* and *cache pollution* is evident in the graph.

For example, consider the smallest (1KB) cache in Graph #4. Increasing block size from 16 B to 32 B is helpful (reduces miss rate) due to exploiting more spatial locality >. But then increasing block size further, from __32_ B to __128_ B, is not helpful (increases miss rate) due to cache pollution having greater effect.

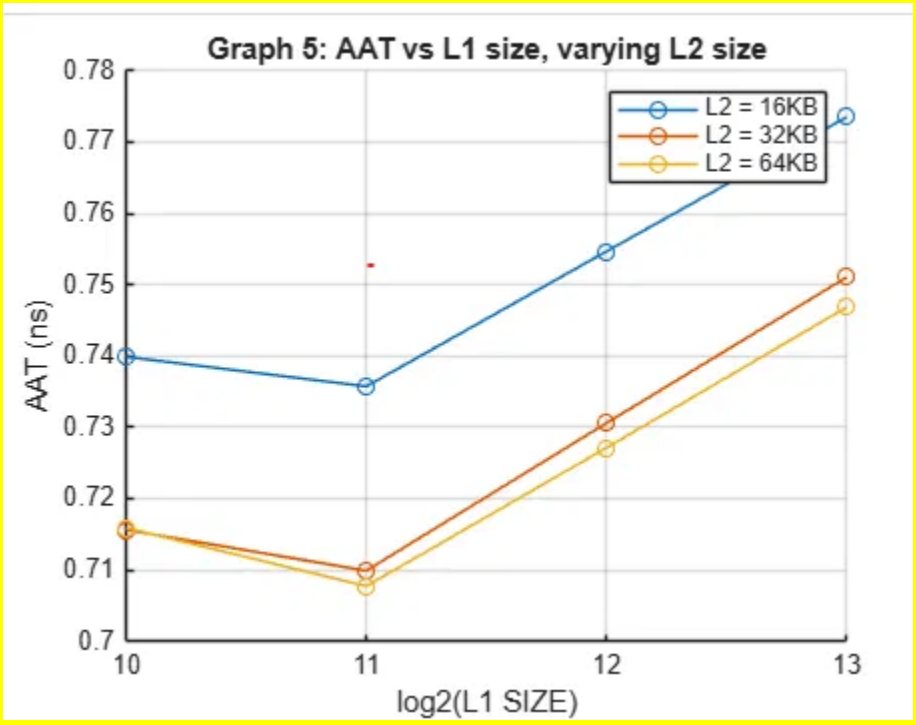
3. L1 + L2 co-exploration

GRAPH #5 (***total number of simulations: 12***)

For this experiment:

- Benchmark trace: gcc_trace.txt
- L1 cache: SIZE is varied, BLOCKSIZE = 32, ASSOC = 4.
- L2 cache: SIZE is varied, BLOCKSIZE = 32, ASSOC = 8.
- Prefetching: None.

Plot AAT on the y-axis versus $\log_2$(L1 SIZE) on the x-axis, for four different L1 cache sizes: L1 SIZE = 1KB, 2KB, 4KB, 8KB. (That is, $\log_2$(L1 SIZE) = 10, 11, 12, 13.) The graph should contain three separate curves (*i.e.*, lines connecting points), one for each of the following L2 cache sizes: 16KB, 32KB, 64KB. All points for the 16KB L2 cache should be connected with a line, all points for the 32KB L2 cache should be connected with a line, *etc*.



Answer the following question:

1. Which memory hierarchy configuration in Graph #5 yields the best (*i.e.*, lowest) AAT and what is that AAT?

Configuration that yields the lowest AAT: ___L1 = 2 KB, 4-way, 32 B blocks; L2 = 64 KB, 8-way, 32 B blocks _____
Lowest AAT: _____0.7075 ns _____

4. Stream buffers study (ECE 563 students only)

TABLE #1 (*total number of simulations: 5*)

For this experiment:

- Microbenchmark: stream_trace.txt
- L1 cache: SIZE = 1KB, ASSOC = 1, BLOCKSIZE = 16.
- L2 cache: None.
- PREF_N (number of stream buffers): 0 (pref. disabled), 1, 2, 3, 4
- PREF_M (number of blocks in each stream buffer): 4

The trace "stream_trace.txt" was generated from the loads and stores in the loop of interest of the following microbenchmark:

```
#define SIZE 1000

uint32_t a[SIZE];
uint32_t b[SIZE];
uint32_t c[SIZE];

int main(int argc, char *argv[]) {
…
   // LOOP OF INTEREST
   for (int i = 0; i < SIZE; i++)
      c[i] = a[i] + b[i];   // per iteration: 2 loads (a[i], b[i]) and 1 store (c[i] = …)
…
}
```

PREF_N	miss rate	prefetches	traffic
0	0.2500	0	994
1	0.2500	3000	3994
2	0.2500	3000	3994
3	0.0010	759	1006
4	0.0010	759	1006

TABLE 5

1. For this streaming microbenchmark, with prefetching disabled, do L1 cache size and/or associativity affect the L1 miss rate (feel free to simulate L1 configurations besides the one used for the table)? Why or why not?

With prefetching disabled, L1 cache size and/or associativity << do not >> affect L1 miss rate (for this streaming microbenchmark).

The reason: _____Capacity and conflict misses only arise when a block needs to survive long enough to be re-referenced, so with no temporal reuse there are none to eliminate. Every miss is compulsory, and the compulsory miss count is fixed by block size alone, not by capacity or associativity.

2. For this streaming microbenchmark, what is the L1 miss rate with prefetching disabled? Why is it that value, *i.e.*, what is causing it to be that value? Hint: each element of arrays a, b, and c, is 4 bytes (uint32_t).

The L1 miss rate with prefetching disabled is ____0.25_____, because each 16 B block holds 4 array elements (16 ÷ 4 bytes). Traversing sequentially, the first element of a block misses and the next three hit, giving exactly 1 miss per 4 accesses = 0.25..

3. For this streaming microbenchmark, with prefetching disabled, what would the L1 miss rate be if you doubled the block size from 16B to 32B? (hypothesize what it will be and then check your hypothesis with a simulation)

The L1 miss rate with prefetching disabled and a block size of 32B is _____0.125_____, because a 32 B block holds 8 elements (32 ÷ 4 bytes), so one miss serves 8 accesses: 1/8 = 0.125. The measured value is marginally higher (378 misses rather than 375) because the arrays are not 32-byte aligned, so each of the three arrays straddles one extra block boundary.

4. With prefetching enabled, what is the minimum number of stream buffers required to have any effect on L1 miss rate? What is the effect on L1 miss rate when this many stream buffers are used: specifically, is it a modest effect or huge effect? Why are this many stream buffers required? Why is using fewer stream buffers futile? Why is using more stream buffers wasteful?

Minimum number of stream buffers needed to have any effect on L1 miss rate: ___3__

With this many stream buffers, the effect on L1 miss rate is <u><< huge >></u>. Specifically, the L1 miss rate is nearly __0___. We only miss on the first elements of __the three arrays a,b,c_____ (hence a total of ____3__ misses).

This many stream buffers are required because the loop interleaves three independent address streams and each stream buffer tracks exactly one stream

Using fewer stream buffers is futile because With N < 3, each miss allocates the LRU buffer to a new stream, which the next miss belonging to a different array immediately steals back. No buffer ever survives until the next reference to its own stream, so there are zero hits. The miss rate stays at 0.25 while memory traffic quadruples (994 → 3994) on prefetches that are never used.

Using more stream buffers is wasteful because _____only three streams exist, so a fourth buffer is never allocated. N = 4 produces identical results to N = 3 (same miss rate, same 759 prefetches, same traffic) while costing extra hardware.
_____.